

Paper Critique: SWE-agent and AutoCodeRover

Oliver Browne

Papers:

- Yang et al., *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*, NeurIPS '24
- Zhang et al., *AutoCodeRover: Autonomous Program Improvement*, ISSTA '24

Both papers try to fix real GitHub issues from SWE-bench. SWE-agent gives the model a custom interactive environment. AutoCodeRover gives it program-analysis tools inside a fixed workflow. I also ran both tools with a small local model (Qwen2.5-Coder 1.5B), which can't confirm their scores but does show which parts hold up.

1 What I liked

Both papers pick the problem developers actually face: fixing real bugs in large repositories. SWE-agent shows that changing only the interface moves results by several points, and its linter rejected a broken edit in my run. AutoCodeRover searches code by its structure, and in my run it found the right `split` method where `grep` would have returned two. Maintainers with long backlogs could use this now for simple bugs, though handing an agent any ticket is still years away. The controversial part is that each paper credits a different main cause, and their comparisons use different models.

2 What I disliked

Both papers test almost only on SWE-bench, which has weak tests and may overlap with the training data. Each result comes from a single run, so a gap of one or two points is just a few issues. SWE-agent breaks easily without GPT-4: with my model, the parser dropped the whole body of an edit. AutoCodeRover's search counted four matches for two methods and crashed on Windows. The released code has also drifted from both papers, and AutoCodeRover exited with code 0 after an error.

3 Future directions

I'd combine the two ideas by giving SWE-agent's interface AutoCodeRover's structural search as commands. The interface should also reject badly formatted commands so that small local models can use it. The same tools could help with security fixes or API migrations. Evaluations need newer repositories, other languages, repeated runs and cost per fix. A study of whether maintainers accept these patches would show if they save real time.

4 Open questions

How much of SWE-agent’s gain comes from GPT-4 already knowing the shell? When AutoCodeRover misses the bug, is the search tool wrong or the model’s query? If a patch passes the tests but differs from the developer’s fix, is it wrong? How do these tools handle bugs with no clear location, such as performance or concurrency problems? Are issue-to-patch tasks typical of real maintenance work?

5 Most important for class discussion

I want to discuss where the gain really comes from: the interface, the analysis or the model. What experiment would separate them? We should also ask how far to trust SWE-bench scores. Neither tool produced a patch in my runs, so I’d like to talk about how reviewers should judge artifacts that stop working over time. I’d also like the professor’s view on whether industry is moving toward open-ended agents or fixed pipelines.

6 Summary

Both papers show that LLMs fix bugs better when they get good tools. SWE-agent improves the interface, and AutoCodeRover brings in program analysis. Neither replaces a strong model. With my small model, neither tool fixed an easy bug, and some of the failures came from the tools themselves.